# Introdução à Programação Competitiva

Universidade Estadual do Sudoeste da Bahia - UESB

2026.1

# Longest Bitonic Subsequence

**Problem Statement:** Given an array `arr` of `n` integers, the task is to find the length of the longest bitonic sequence. A sequence is considered bitonic if it first increases, then decreases. The sequence does not have to be contiguous.

**Example 1:**
Input: `arr = [5, 1, 4, 2, 3, 6, 8, 7]`
Output: 6
Explanation: The longest bitonic sequence is `[1, 2, 3, 6, 8, 7]`. It increases from 1 to 8 and then decreases at 7.

**Example 2:**
Input: `arr = [10, 20, 30, 40, 50, 40, 30, 20]`
Output: 8
Explanation: The entire array is bitonic. Length is 8.

# Processo Seletivo: Estágio em Engenharia de Software

# Como resolver

- **Força bruta**: Testa todas as combinações.
- Funciona para 10 números.
- Trava o servidor para entradas muito grandes.
- Solução: **Programação Dinâmica**.

# O Motivo

- **Fundamentos:** Ferramentas mudam; memória e processamento, não.
- **Perfil Lógico:** Capacidade de resolver problemas complexos sob pressão.

# O Esporte da Mente

Dividido em duas etapas:

1. **Design do Algoritmo:** Matemática, lógica e restrições.
2. **Implementação:** Código rápido, limpo e sem bugs.

**Efeitos colaterais:** Agilidade mental, domínio de Estruturas de Dados, debugging rápido e trabalho sob pressão.

# Juízes Online

- Codeforces
- CSES
- Kattis
- AtCoder
- CodeChef

# Como ler um desafio

- **Descrição:** O contexto.
- **Entrada:** O formato exato dos dados que você vai receber.
- **Saída:** O formato exato que o juiz espera.
- **Exemplos:** Casos de teste básicos.
- **Limites:** Tempo (ex: $1.0s$) e Memória (ex: $256MB$).

# Feedbacks das questões

- **Accepted (AC):** Certo e rápido.
- **Wrong Answer (WA):** Lógica errada.
- **Compilation Error (CE):** Erro de sintaxe.
- **Runtime Error (RE):** Código quebrado (ex: divisão por 0).
- **Memory Limit Exceeded (MLE):** Estourou a RAM.
- **Time Limit Exceeded (TLE):** Lógica certa, algoritmo lento.

# Como Treinar

- **Padrões:** Estude tipos comuns de problemas.
- **Teoria:** Domine Estruturas de Dados e Algoritmos clássicos.
- **Prática:** Resolução implacável. E depois, mais prática.

# Dicas Importantes

- **Rapidez:** Digite rápido.
- **Classifique:** Pense no padrão antes de escrever o código.
- **Complexidade:** Calcule se vai dar TLE de cabeça.
- **Teste:** Crie casos extremos (negativos, zero, limite máximo).

# Aprenda a desistir

- Travar faz parte.
- Se você passou de 20 a 40 minutos sem **nenhum progresso significativo**, abra o Editorial.
- Não olhe o código direto. Leia a explicação passo a passo até destravar e tente voltar a programar sozinho.
- Se você leu a solução para entender a ideia, você **obrigatoriamente** precisa implementar o código sozinho depois.

# Expanda seu repertório

- Leia códigos de terceiros, veja como eles fizeram mais rápido ou com menos linhas.
- Simule a pressão fazendo provas passadas com o cronômetro rodando. Isso treina sua consistência e gerenciamento de tempo.
- Aprenda Lendo.

# Ferramentas Úteis

- **USACO Guide / CP-Algorithms:** Roteiros de estudo e catálogos completos de algoritmos.
- **CS Academy:** Excelente ferramenta visual. Possui um *Graph Editor* para desenhar grafos e testar fluxos visuais.
- **ACodeDaily:** Plataforma para rastrear seu progresso e resolver exercícios selecionados por tópico ou nível de dificuldade.
- **VJudge:** Onde faremos nossos treinos. Ele unifica problemas de todos os outros juízes em um só lugar.

# Template C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    // sua solucao vem aqui
    return 0;
}
```

*Nota: `bits/stdc++.h` importa tudo. Na maratona, tempo importa mais que limpeza de escopo.*

# Compilando Localmente

```
g++ -std=c++11 -O2 -Wall test.cpp -o test
```

- `-Wall`: Liga avisos de erros comuns.
- `-O2`: Otimiza a velocidade do código.

# Otimização de I/O

```
ios_base::sync_with_stdio(0);
cin.tie(0);
```

- Desliga a sincronização do C++ com o C.
- Nunca use `endl`. Use `\n`. O *flush* do `endl` é pesado.

# Integer Overflow

- `int`: 32 bits (aguenta até $\approx 2 \times 10^9$)
- `long long`: 64 bits (aguenta até $\approx 9 \times 10^{18}$)

**Exemplo:** Se multiplicar $10^5 \times 10^5$, o `int` já não serve. Na dúvida? Use `long long`.

# Macros e Typedefs

```
typedef long long ll;
#define PB push_back
```

```
ll x = 5;
vetor.PB(10);
```

**Aviso:** Use para agilizar o que é repetitivo, não para criar um código ilegível.

# Hora de Codar

1. Crie conta no VJudge.
2. Acesse o Contest da aula.

**Problemas de Hoje:**

- `Cold-puter Science` (Kattis)
- `Soldier and Bananas` (Codeforces)
- `Weird Algorithm` (CSES)

# Detalhes da disciplina

- **Tópicos:**
  - **Unidade I:** STL, Ordenação.
  - **Unidade II:** Busca Binária, Algoritmos, Gulosos, Teoria dos Números.
  - **Unidade III:** Grafos, DP.
- **Metodologia:** Laboratório prático.

# Accepted

Tempo: 0.00s  Memória: 0 KB

**Fim do Dia 1.**

Nos vemos na próxima aula.